AP HOGESCHOOL
ANTWERPEN

Het web als applicatieplatform

Web Frameworks · Les 1

Kristof Michiels · labo's: Sven Mariën, Kristof Michiels en Andie Similon

Vandaag

- waarom dit vak, en hoe we met AI werken
- van document naar toepassing: frontend vs backend
- wat verstaan we onder framework?
- het landschap: van vanilla JS tot Angular
- herhaling: TypeScript en Node
- je gereedschap: de Angular CLI

Alles staat uitgeschreven in **hoofdstuk 1 van de syllabus**. De nummers op de slides (1.2, 1.4, ...) verwijzen ernaar.

Praktisch

- hoorcollege om de twee weken, labo elke week
- week 1: les 1 en labo 1 - week 2: labo 2 - week 3: les 2
- elk hoofdstuk van de syllabus draagt twee labo's
- examen: een **vaardigheidstoets** van 3 uur, zonder AI
  je werkt een gestripte toepassing af tot ze doet wat de demo-video toont
  je toont je een **bekwame junior Angular ontwikkelaar**. Really!
- referenties: de syllabus en **angular.dev**

Waarom dit vak?

1.1

Het sluitstuk van de drie webvakken

VAK	VERWORVEN KENNIS
Web Technology	HTML, CSS, de DOM
Web Programming	JavaScript en TypeScript, Node, npm, Express, MySQL, Fetch API
Web Frameworks	alles samen: volwaardige toepassingen, van scherm tot database

Nieuw is vooral de **schaal**, en de tools dat die schaal beheersbaar maken.

Waarom nog zelf code schrijven?

Code produceren is goedkoop geworden.
Code begrijpen, beoordelen en corrigeren is dat niet.

- frameworks zijn de taal waarin AI-modellen frontend-code genereren
- **AI-modellen genereren massaal verouderde Angular-code** 😒
  Angular veranderde grondig tussen 2023 en 2026; de trainingsdata is vaak ouder
- zien wat er niet klopt, is een kernvaardigheid van een moderne developer

Conclusie: je schrijft je code zelf 😉

Handmatig kunnen bouwen is het bewijs dat het model in je hoofd zit.
En het examen is zonder AI.

Hoe we met AI werken

LABEL	BETEKENIS
zonder AI	je gebruikt geen AI; zo is ook de vaardigheidstoets
zonder AI aanbevolen	je mag AI gebruiken; dit staat op de meeste drills
met AI	AI-output beoordelen en moderniseren; we oefenen je analytische skills

- AI mag je helpen *begrijpen*, niet *produceren*
- de proef op de som: elke regel die je indient, kan je mondeling uitleggen

Van document naar toepassing

1.2

Het web van documenten

- de browser vraagt een HTML-bestand, de server stuurt het, de browser toont het
- elke klik begint die conversatie opnieuw
- voor een artikel, een portfolio, een nieuwssite nog altijd het juiste model

Maar Gmail, Spotify, je online bank, ons Digitap leerplatform zijn geen documenten.
Het zijn **toepassingen** die in een browser draaien.

Waar wordt de HTML gemaakt?

Eén vraag die de webarchitectuur verklaart.

Twee antwoorden

	SERVER-SIDE TOEPASSING	SPA MET API
Waar ontstaat de HTML?	op de server, per aanvraag	in de browser, door JavaScript
Wat gaat er over het netwerk?	volledige pagina's	alleen data (JSON)
Na een klik?	nieuwe pagina, alles herlaadt	een stuk van het scherm verandert
Waar leeft de state?	op de server (sessie)	in de browser
Wat doet de server?	alles	data bewaren en beslissen

PHP, Express met templates, Flask vs Angular, React, Vue

Frontend, backend, API

Frontend

alles wat in de browser draait
wat de gebruiker ziet en aanraakt
Angular · hoofdstuk 2 tot 5

Backend

alles wat op de server draait
bewaren, regels afdwingen, beslissen
Nest en MongoDB · hoofdstuk 6

Ertussen: de **API**, het contract over welke data heen en weer gaat, en in welke vorm.

Voordeel van de opdeling?

- **een vlottere gebruikerservaring**: de code die reageert, zit al in de browser
- **schalen wordt goedkoper**: zie volgende slide
- **één backend, meerdere frontends**: web, mobiel, partners
- **teams werken los van elkaar**: de API is het contract

Schalen: wie doet het werk?

Server-side

- de server bouwt elke pagina, voor elke gebruiker, bij elke klik
- 1000 gebruikers = 1000 keer dat werk, op jouw server

SPA met API

- 1000 gebruikers brengen 1000 processoren mee
- de frontend is statisch: een eenvoudige webserver of CDN volstaat
- de backend levert kleine JSON-antwoorden en onthoudt geen schermen: zet er gewoon een exemplaar bij

Nadelen van het model?

- de eerste keer laden duurt langer
- moeilijker indexeerbaar door zoekmachines (er zijn oplossingen)
- zonder JavaScript is er niets
- alles in de browser is in handen van de gebruiker: de backend vertrouwt de frontend nooit (hoofdstuk 7)
- de browser moet nu **zelf bijhouden wat er op het scherm staat**

Dat laatste behoort tot de kern van waar dit vak om draait.

Zes belangrijke verschuivingen in dertig jaar

VANAF	VERSCHUIVING	NAMEN
1991	statische documenten	HTML, later CSS
1995	de server bouwt pagina's uit een database	PHP, ASP, JSP; later Rails, Django, Express
2005	AJAX: data ophalen zonder herladen	Gmail, Google Maps, jQuery
2010	de toepassing verhuist naar de browser	Backbone, AngularJS, Ember; Node.js
2013	componenten en declaratieve UI	React, Vue, Angular (2016); TypeScript
2020	de slinger zwaait terug, en wordt fijner	Next.js, Astro, HTMX; signals

Een slinger tussen server en client. Dit vak: rij vier en vijf, met de reactiviteit van rij zes.
En de zevende is bezig: sinds 2022 schrijft AI een groeiend deel van alle code.

State en het scherm

- **state**: de gegevens zoals ze op dit moment zijn
  de taken, de ingelogde gebruiker, het winkelmandje, de filter
- in je code: gewoon een variabele
- het scherm **toont** die state, maar **is** die state niet
- de DOM is een tweede, aparte voorstelling van dezelfde informatie

Verander je de array, dan verandert er op het scherm niets.

In vanilla JS verzorg jij de synchronisatie

```
const taken = [];

function render() {
  const lijst = document.querySelector('#taken');
  lijst.innerHTML = '';
  for (const taak of taken) {
    const li = document.createElement('li');
    li.textContent = taak.titel;
    lijst.append(li);
  }
}

function voegToe(titel) {
  taken.push({ titel, gedaan: false });  // 1. change the data
  render();                              // 2. remember to update the screen!
}
```

Het synchronisatieprobleem

- vergeet één `render()` en je toepassing toont oude informatie
- **zonder foutmelding**
- één lijstje hou je vol; een echte toepassing heeft tientallen stukken state, elk op meerdere plaatsen zichtbaar
- dé bron van bugs in klassieke DOM-scripting

Een framework is in essentie een machine die het scherm automatisch synchroon houdt met je state.

Componenten, routing, forms en tooling zijn ondersteunende tools rond die kern.

Wat een framework is

1.3 en 1.4

Library of framework?

	LIBRARY	FRAMEWORK
Wie roept wie aan?	jouw code roept de library aan	het framework roept jouw code aan
Wie bepaalt de structuur?	jij	het framework
Voorbeeld	`fetch`, een datumbibliotheek	Angular, Nest

Inversion of control: jij levert onderdelen aan; het framework beslist wanneer ze draaien. Jij beschrijft *wat* er moet bestaan. *Wanneer* en *hoe vaak* is voor de machine.

Imperatief vs declaratief

Imperatief: hoe

```
lijst.innerHTML = '';
for (const taak of taken) {
  const li = document
    .createElement('li');
  li.textContent = taak.titel;
  lijst.append(li);
}
```

Declaratief: wat

```
<ul>
  @for (taak of taken(); track taak.id) {
    <li>{{ taak.titel }}</li>
  }
</ul>
```

Stap twee bestaat niet meer, en kan dus niet meer vergeten worden.

De UI is een beschrijving, geen reeks handelingen.

Wat je erbij krijgt, en wat het kost

- een framework is **opinionated**: waar bestanden staan, hoe onderdelen heten, hoe data stroomt
- voor een eenvoudige landingspagina: overkill
- voor een team en een toepassing die groeit: elke Angular-developer vindt zijn weg in elk Angular-project
- de prijs: een leercurve, en die van Angular is steil. Onderschat het niet. Het docententeam doet zijn best, jij ook
- de begrippen zijn niet Angular-specifiek: componenten, templates, reactiviteit, dependency injection, routing => kennis herbruikbaar voor andere frameworks

Geen magie: het doosje

```
let actiefEffect = null;

export function signal(waarde) {
  const luisteraars = new Set();
  return {
    get() {
      if (actiefEffect) luisteraars.add(actiefEffect); // remember who reads
      return waarde;
    },
    set(nieuw) {
      waarde = nieuw;
      luisteraars.forEach(fn => fn());                 // notify all readers
    },
  };
}
```

Een closure rond een waarde. `get` noteert wie er leest; `set` roept iedereen terug.

Geen magie: wie kijkt erin?

```
export function effect(fn) {
  actiefEffect = fn;  // while fn runs, every get() sees who is asking
  fn();               // run once: this is where subscriptions happen
  actiefEffect = null;
}
```

```
const teller = signal(0);

effect(() => {
  document.querySelector('#uit').textContent = `Je klikte ${teller.get()} keer.`;
});

knop.addEventListener('click', () => teller.set(teller.get() + 1));
```

Er is geen stap twee meer. **In labo 1 bouw je dit zelf.**

Wie een signal leest, wordt onthouden. Verandert het signal, dan wordt die code opnieuw uitgevoerd.

Wat Angular daar nog bovenop doet: berekeningen onthouden met "computed", meerdere updates slim samen uitvoeren, automatisch opruimen en veel werk via de compiler regelen.

Het landschap

1.5, 1.6 en 1.7

Het spectrum: van strooisel tot platform

Hoeveel van de verantwoordelijkheid geef je uit handen?

Vanilla
jij doet alles, ook de synchronisatie

Lightweight
Alpine.js, HTMX: attributen op bestaande HTML

SPA-frameworks
React, Vue, Angular: de toepassing leeft in de browser

Meta-frameworks
Next.js, Nuxt, Astro, Analog: de server doet opnieuw mee

Dit vak: SPA met client-side rendering. Voor dashboards, beheertools en apps achter een login is dat het juiste antwoord.

Alpine.js: dezelfde begrippen, in het klein

```
<div x-data="{ taken: [], nieuw: '' }">
  <input x-model="nieuw" placeholder="Nieuwe taak">
  <button @click="taken.push(nieuw); nieuw = ''">Toevoegen</button>
  <ul>
    <template x-for="taak in taken">
      <li x-text="taak"></li>
    </template>
  </ul>
  <p x-show="taken.length === 0">Nog geen taken.</p>
</div>
```

State, two-way binding, event binding, een lus, een conditie: de begrippen van hoofdstuk 2. Wat ontbreekt: componenten met contracten, types, routing, tooling, structuur voor een team.

HTMX: de server blijft de baas

```
<button hx-get="/taken/lijst" hx-target="#lijst">Ververs</button>
<div id="lijst"><!-- server-rendered rows land here --></div>
```

- de server stuurt fragmenten HTML, geen JSON; geen state in de browser
- een terechte kritiek op de SPA voor sites die vooral document zijn
- waarom niet in dit vak? Veel state in de browser vraagt beheer in de browser. En HTMX heeft een server nodig; die bouw je pas in hoofdstuk 6

Niet elke interactieve site hoort een SPA te zijn.

Een engineeringbeslissing

- hoeveel state heeft dit product in de browser?
- hoeveel schermen?
- hoe lang moet het meegaan?
- hoeveel mensen werken eraan?

Een portfoliosite in Angular: met een vrachtwagen naar de bakker.
Een bankapplicatie op los strooisel: de vrachtlading op een bakfiets.

De grote drie

	SINDS	KARAKTER
React (Meta)	2013	een library voor de UI-laag; de rest kies je zelf; op dit moment het populairst
Vue	2014	het progressieve midden; lage instapdrempel
Angular (Google)	2016	het volledige platform: router, forms, HTTP, testen, CLI, in TypeScript

- ze groeien naar elkaar toe: componenten, declaratieve UI, fijnmazige reactiviteit
- waarom Angular? Een opleiding traint modellen, geen syntax. Angular is het best leerbare en toetsbare voertuig
- wie Angular grondig begrijpt, leest en schrijft React binnen een week

Angular vandaag

2010	**AngularJS**: vernieuwend, maar een doodlopende straat
2016	**Angular 2**: van nul herschreven; sindsdien elk half jaar een major versie
2023	v16: de **Angular Renaissance** begint. NgModules overbodig, ingebouwde control flow, **signals**
2025	v21: nieuwe projecten draaien zoneless
2026	**Angular 22**: de versie van deze syllabus

Het internet weet dit nog niet. Tutorials, Stack Overflow en AI-modellen lopen achter.

Vuistregel: jouw referentie is angular.dev en de syllabus. Al de rest lees je met een blik op de datum.

Herhaling met een nieuwe rol

1.8 en 1.9

TypeScript: types als contract

```ts
interface Gebruiker {
  id: number;
  naam: string;
  email: string;
}

const gebruiker: Gebruiker = { id: 1, naam: 'Barbara', email: 'barbara@ap.be' };
console.log(gebruiker.name);
// error TS2339: Property 'name' does not exist on type 'Gebruiker'.
```

De fout verhuist van runtime (bij de gebruiker) naar compile time (in je editor).

Drie eigenschappen om te onthouden

- **lui op de goede manier**: de compiler leidt types af waar hij kan
  je schrijft ze waar ze een contract vormen: datamodellen, functiesignaturen
- **vorm, niet naam**: wie de juiste velden heeft, *is* een `Gebruiker`
  ideaal voor JSON die van een API komt
- **types bestaan alleen tijdens het compileren**
  een interface beschermt je tegen je eigen typfouten, niet tegen een API die iets anders stuurt

Node: de werkplaats van de frontend-developer

- je toepassing draait in de browser; je **gereedschap** draait op Node
  de CLI, de dev server, de TypeScript-compiler
- Angular 22 vraagt minstens Node 22.22, of 24.15 en hoger
- **Vuistregel:** installeer de huidige LTS (Node 24)

```
{
  "scripts": { "start": "ng serve", "build": "ng build" },
  "dependencies": { "@angular/core": "^22.1.0" },
  "devDependencies": { "@angular/cli": "^22.1.8" }
}
```

`dependencies` om te draaien · `devDependencies` om te bouwen · `^` = binnen versie 22

`node_modules`: twee regels

- **wegwerpmateriaal**: staat in `.gitignore`, gaat nooit mee in een zip of een repository
- zie je een project zonder `node_modules`? Dan is `npm install` je eerste commando

Bijna 300 MB op jouw schijf. Wat de gebruiker downloadt: een goede 50 kB.

Je gereedschap: de Angular CLI

1.10 · de handleiding bij labo 2

Van niets tot een draaiende toepassing

```
npm install -g @angular/cli    # once
ng version

ng new eerste-app              # CSS · no SSR · no AI config
cd eerste-app
ng serve --open                # http://localhost:4200
```

De werklus voor de rest van het semester: **bewerk, bewaar, kijk.**

In VS Code: de extensie **Angular Language Service** is verplicht.

Wat de CLI voor je maakte

HET PROJECT

`package.json`	het paspoort
`node_modules/`	wegwerpmateriaal
`angular.json`	configuratie van de CLI
`tsconfig.json`	de TypeScript-compiler

DE INGANG

`src/index.html`	de ene pagina van je SPA
`src/main.ts`	geeft de controle aan Angular
`src/styles.css`	globale stijlen

JOUW DOMEIN: `SRC/APP/`

`app.ts`	de rootcomponent: klasse
`app.html`	de rootcomponent: template
`app.css`	de rootcomponent: stijlen
`app.spec.ts`	test; laat staan
`app.config.ts`	vanaf hoofdstuk 3
`app.routes.ts`	vanaf hoofdstuk 3

Al de rest mag je dit semester negeren.

Twee bestanden, twee inzichten

```
<!-- src/index.html -->
<body>
  <app-root></app-root>
</body>
```

Bijna leeg, en toch een volledige pagina: alles is door JavaScript opgebouwd. Dat is de **SPA**.

```
// src/main.ts
bootstrapApplication(App, appConfig);
```

Jouw programma start niet. Het geeft zijn rootcomponent af aan het framework. Dat is **inversion of control**.

Een eerste blik op een component

```
// src/app/app.ts
@Component({
  imports: [RouterOutlet],
  selector: 'app-root',
  styleUrl: './app.css',
  templateUrl: './app.html',
})
export class App {
  protected readonly title = signal('eerste-app');
}
```

Een klasse (wat het weet en kan), een template (wat er staat), stijlen (hoe het eruitziet).

Een Angular-toepassing is een **boom van componenten**, met `App` als root element.

Hoofdstuk 2 gaat in de diepte.

`ng generate`: een component maken en plaatsen

```
ng g c begroeting
```

```
<!-- 1. app.html: put the tag in the parent's template -->
<app-begroeting />
```

```
// 2. app.ts: import the class in the parent
import { Begroeting } from './begroeting/begroeting';

@Component({ imports: [RouterOutlet, Begroeting], /* ... */ })
```

Vergeet je stap twee, dan weigert de compiler met **NG8001**. In labo 2 maak je die fout één keer bewust.

De commando's die je moet kennen

COMMANDO	WAT HET DOET
`ng new <naam>`	een nieuw project genereren
`ng serve`	de dev server starten (`--open`, `--port 4300`)
`ng g c <naam>`	een component genereren
`... --dry-run`	tonen wat er zou gebeuren, zonder iets te schrijven
`ng build`	bouwen voor productie, in `dist/`
`ng <commando> --help`	de ingebouwde handleiding

Niemand kent de CLI uit het hoofd. Ze documenteert zichzelf.

Gedateerde Angular-code herkennen

Zie je één van deze patronen, dan lees je oude code:

- een bestand `app.module.ts` of `@NgModule({...})`
- `*ngIf` of `*ngFor` in templates · modern: `@if`, `@for`
- bestandsnamen als `app.component.ts` · modern: `app.ts`
- `standalone: true` in elke component
- uitleg die over `zone.js` begint

Jouw project is correct; de bron is oud.

Waar dit vak naartoe werkt

- hoofdstuk 2 tot 5: de **frontend**, in Angular
- hoofdstuk 6: de **backend**, met Nest en MongoDB
  Nest is bewust naar Angular gemodelleerd; je Express-kennis komt van pas
- hoofdstuk 7: afwerken, met Material en security
- alles in TypeScript, van formulierveld tot databasequery

Deze week en volgende week

Labo 1 · Geen magie

- de takenlijst in vanilla
- bug hunt (**zonder AI**)
- je eigen `signal` en `effect`
- dezelfde app in Alpine
- je werkplaats installeren

Labo 2 · De Angular CLI

- `ng new`, `ng serve`, `ng build`
- elk gegenereerd bestand kennen
- je eerste componenten

Lees vooraf **syllabus 1.10**.

Week 2: geen theorie. Les 2 (componenten en reactiviteit) volgt in week 3.